

Title: Study of Digital-to-Digital Conversion (Line Coding) Using MATLAB

Objectives:

1. To study the concept of Digital-to-Digital Conversion using Line Coding in MATLAB.
2. To observe different line coding techniques and signal representation.
3. To analyze how digital data is converted into digital signals using MATLAB simulation.
4. To compare different line coding schemes and their signal characteristics.

Theory:

Digital-to-digital conversion is a process in which digital data is converted into digital signals for transmission over a communication channel. This process is commonly known as line coding. In communication systems, data stored as bits (0s and 1s) must be converted into signals so that it can be transmitted from the transmitter to the receiver.

Line Coding:

Line coding is the method of representing binary data using digital signal levels. Different techniques are used to encode the bits into signals. At the transmitter side, digital data is encoded into a signal, and at the receiver side the signal is decoded to recover the original data.

Signal and Data Elements:

A data element represents the smallest unit of information (a single bit). A signal element is the smallest unit of the digital signal used to carry the data. In digital communication, signal elements are used to transmit data elements.

Bandwidth:

Digital signals are usually non-periodic and theoretically have unlimited bandwidth. However, in practical communication systems only a limited range of frequencies is used. This is called effective bandwidth, which is important for efficient data transmission.

Line Coding Techniques:

Various line coding schemes such as Polar NRZ-I, Differential Manchester, and AMI are used in digital communication. These techniques differ in signal representation, synchronization capability, and bandwidth usage.

Code:

1) Polar NRZ-I

```
%ID = 23-54972-3
% AB-CDEFG-H , EFG = 972, Binary bit = 1001 0111 0010
% Polar NRZ-I
clc;
clear all;
bit_stream = [1 0 0 1 0 1 1 1 0 0 1 0];
no_bits = length(bit_stream); %no_bit=12
bit_rate = 1000; % 1 kbps
pulse_per_bit = 1; % for polar nrz
pulse_duration = 1/((pulse_per_bit)*(bit_rate)); %pulse_duration=1/1*1000=1x10-3 (second)=1 ms
no_pulses = no_bits*pulse_per_bit; %no_pulses=12*1=12

samples_per_pulse = 500;
fs = (samples_per_pulse)/(pulse_duration); %sampling frequency =500/1ms=50000
t = 0:1/fs:(no_pulses)*(pulse_duration); % sampling interval, total duration = (no_pulse)*(pulse_duration)
no_samples = length(t);
dig_sig = zeros(1,no_samples);

max_voltage = 5;
min_voltage = -5;
inv_bit=1;
last_state=5;
inv_last_state=-5;

for i = 1:length(bit_stream)
    if bit_stream(i) == inv_bit
        dig_sig(((i-1)*(samples_per_pulse)+1):i*(samples_per_pulse)) = inv_last_state*ones(1,samples_per_pulse);
        temp_cons = last_state; % temporary constant
        last_state = inv_last_state;
        inv_last_state = temp_cons;
    else
        dig_sig(((i-1)*(samples_per_pulse)+1):i*(samples_per_pulse)) = last_state*ones(1,samples_per_pulse);
    end
end

plot(t,dig_sig,'linewidth',1.5)
grid on
xticks(0:0.001:0.012)
xlabel('time in seconds')
ylabel('Voltage')
ylim([(min_voltage - (max_voltage)*0.2) (max_voltage+max_voltage*0.2)])
title(['Polar NRZ-I for ',num2str(bit_stream),'])
```

2) Polar Differential Manchester

```
clc;
clear all;
bit_stream = [1 0 0 1 0 1 1 1 0 0 1 0];
no_bits = length(bit_stream);
bit_rate = 1000; % 1 kbps
pulse_per_bit = 2; % for differential manchester
pulse_duration = 1/((pulse_per_bit)*(bit_rate)); %pulse_duration=1/2*1000=1/2000=0.0005=0.5ms
no_pulses = no_bits*pulse_per_bit; % no_pulse=6*2=12

samples_per_pulse = 500;
fs = (samples_per_pulse)/(pulse_duration); %sampling frequency
% including pulse duration in sampling frequency
% ensures having enough samples in each pulse
t = 0:1/fs:(no_pulses)*(pulse_duration); % sampling interval
% total duration = (no_pulse)*(pulse_duration)
no_samples = length(t); % total number of samples
dig_sig = zeros(1,no_samples);

max_voltage = +5;
min_voltage = -5;
inv_bit = 0; % inverting bit (in the lab sheet it is 1, but it should be 0)
last_state = max_voltage; % +v
inv_last_state = min_voltage; % inverse of last state, -v

for i = 1:no_bits
    j = (i-1)*2;
    if bit_stream(i) == inv_bit
        dig_sig((j*(samples_per_pulse)+1):(j+1)*(samples_per_pulse)) = inv_last_state*ones(1,samples_per_pulse);
        dig_sig(((j+1)*(samples_per_pulse)+1):(j+2)*(samples_per_pulse)) = last_state*ones(1,samples_per_pulse);
    else
        dig_sig((j*(samples_per_pulse)+1):(j+1)*(samples_per_pulse)) = last_state*ones(1,samples_per_pulse);
        dig_sig(((j+1)*(samples_per_pulse)+1):(j+2)*(samples_per_pulse)) = inv_last_state*ones(1,samples_per_pulse);
        temp_cons = last_state; % temporary constant
        last_state = inv_last_state;
        inv_last_state = temp_cons;
    end
end
```

3) AMI

```

clc;
clear all;

bit_stream = [1 0 0 1 0 1 1 1 0 0 1 0];
no_bits = length(bit_stream);
bit_rate = 1000; % 1 kbps
pulse_per_bit = 1;
pulse_duration = 1/((pulse_per_bit)*(bit_rate));
no_pulses = no_bits*pulse_per_bit;

samples_per_pulse = 500;
fs = (samples_per_pulse)/(pulse_duration);
t = 0:1/fs:(no_pulses)*(pulse_duration);
no_samples = length(t);

dig_sig = zeros(1,no_samples);
|
max_voltage = 5;
min_voltage = -5;
last_state = -max_voltage;

for i = 1:no_bits
    if bit_stream(i) == 1
        % alternate polarity for each 1
        last_state = -last_state;
        dig_sig(((i-1)*samples_per_pulse+1):(i*samples_per_pulse)) = last_state*ones(1,samples_per_pulse);
    else
        % 0 gives zero voltage
        dig_sig(((i-1)*samples_per_pulse+1):(i*samples_per_pulse)) = 0;
    end
end
end

```

Simulation Figure:

1) Polar NRZ-I

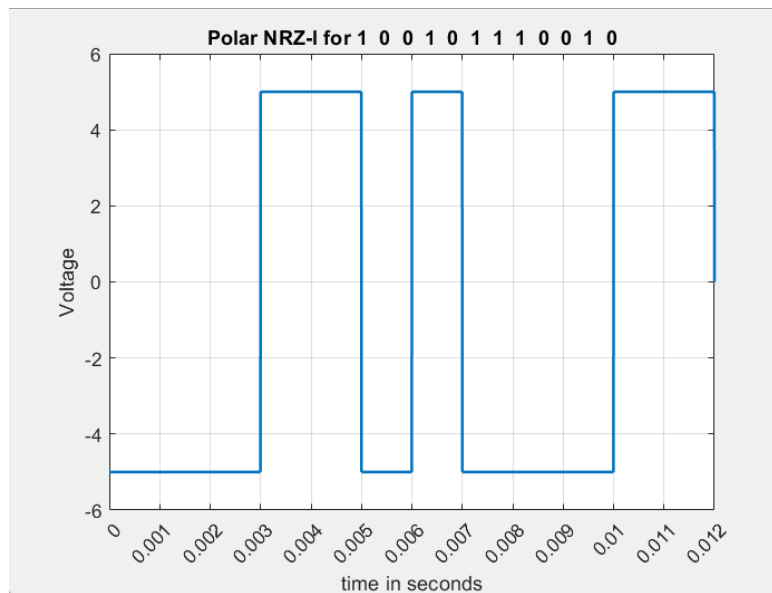


Fig: Polar NRZ-I Signal

2) Polar Differential Manchester

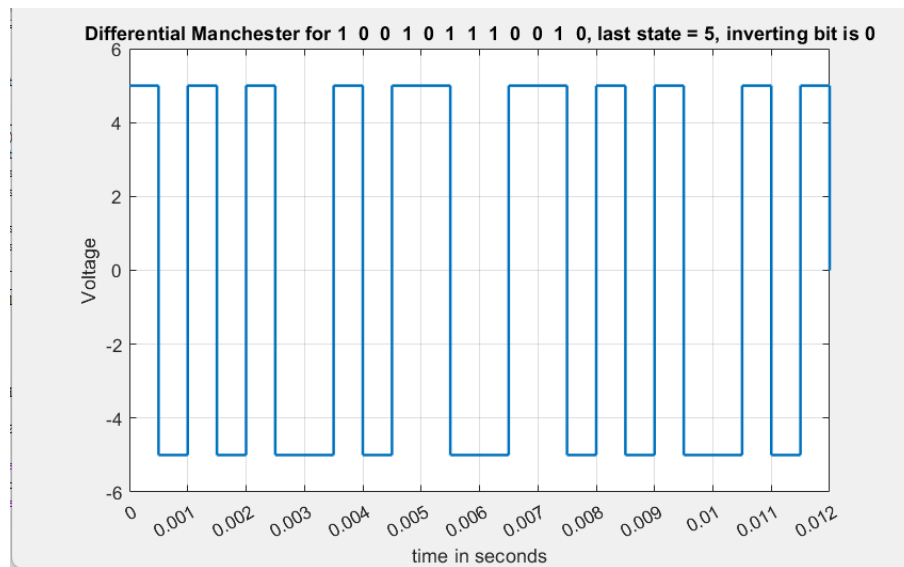


Fig: Manchester Encoded Signal

3) AMI

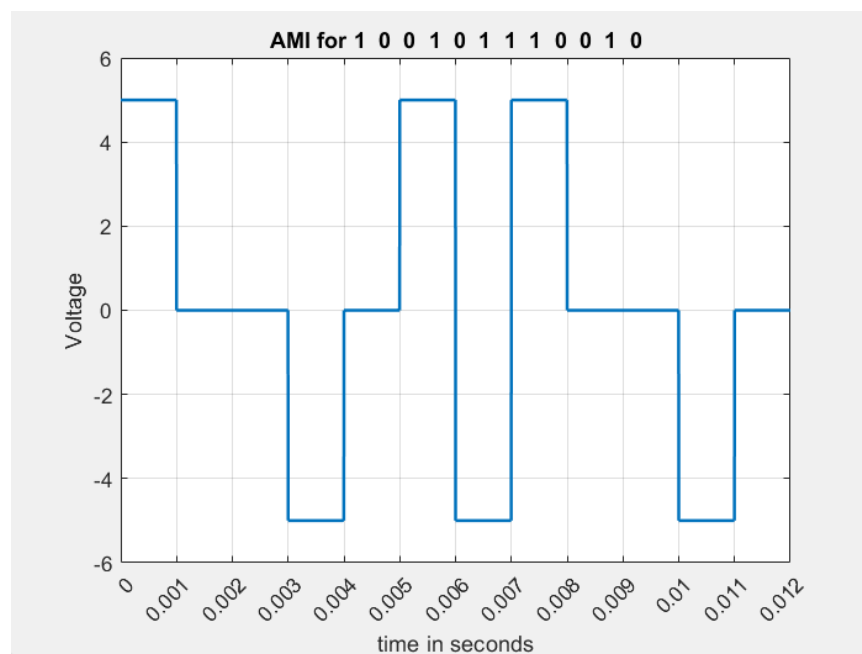


Fig: AMI Encoded Signal

Result analysis/discussion:

In this experiment, different digital-to-digital line coding techniques such as Polar NRZ-I, Polar Differential Manchester, and AMI were simulated using MATLAB. The simulation results showed how binary data is converted into digital signals using different encoding schemes. In the Polar NRZ-I method, a change in signal level represents a binary '1', while no change represents '0', which helps in reducing synchronization problems. In the Differential Manchester scheme, a transition occurs in the middle of every bit period, ensuring better synchronization between transmitter and receiver. The AMI coding scheme represents binary '1' with alternating positive and negative voltages while '0' is represented by zero voltage, which helps in eliminating the DC component and reducing signal distortion. From the simulation waveforms, it was observed that each line coding technique has different signal characteristics and advantages. Therefore, the experiment successfully demonstrated how different line coding schemes affect signal representation and transmission in digital communication systems.

Conclusion:

In this experiment, we successfully implemented and analyzed various digital-to-digital line coding schemes using MATLAB. Each coding scheme has unique characteristics and applications; for example, Manchester encoding ensures synchronization through mid-bit transitions, while AMI eliminates the DC component by alternating the polarity of '1's. MLT-3 efficiently reduces bandwidth requirements by cycling through three voltage levels. These techniques are essential in digital communication systems, where factors such as bandwidth efficiency, synchronization, and baseline wandering need to be considered. Understanding these coding methods provides valuable insights into designing reliable and efficient data transmission systems.

References:

- [1] MathWorks, *MATLAB Documentation*, Natick, MA, USA: MathWorks. [Online]. Available: <https://www.mathworks.com/help/>
- [2] Stormy Attaway, *MATLAB: A Practical Introduction to Programming and Problem Solving*, 6th ed. Boston, MA, USA: Butterworth-Heinemann, 2019.